Programmation Orientée Objet avec Java

Dany Siriphol – CODA 1 ère année - 2026

Origine de la programmation orientée objet (POO)

Crise du logiciel dans les années 70 :

- Augmentation de la puissance des machines
- Logiciels de plus en plus complexes
- Langages complexes et rudimentaires : Assembleur, Fortran, Cobol

Problématique :

- Baisse de la qualité du code
- Explosion des coûts
- Retards
- Logiciels moins fiables

Émergence de l’ingénierie logicielle

"The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software."—**IEEE Standard Glossary of Software Engineering Terminology**

- Gérer la complexité
- Faciliter le travail en équipe
- Maîtriser les budgets

- Améliorer les outils
- Adopter des méthodes et processus de développement pour rationaliser la production d’un logiciel et sa maintenance

Objectifs

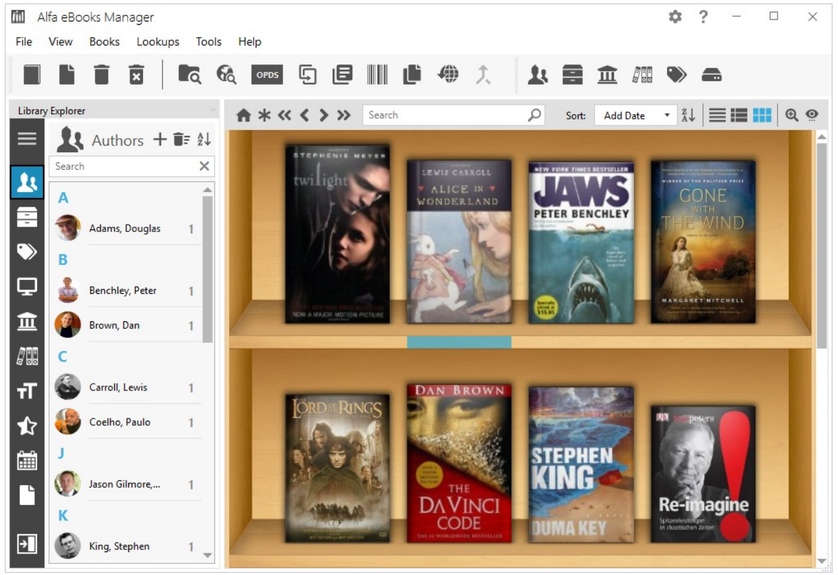
Extensibilité : capacité d’ajouter de nouvelles fonctionnalités facilement

Adaptabilité/Reutilisabilité : capacité à réutiliser un code dans divers contexte

Modularité : créer des codes indépendants, portable et interchangeable

Maintenabilité : Un code maintenable est un code facile à comprendre, modifier et améliorer sans introduire de régressions ou d’erreurs

Adaptabilité



Maintenance

Un logiciel ne s’use pas mais se détériore avec le temps:

- Evolution des technologies
- Changements des besoins du client
- Apparition de bugs

La maintenance peut constituer la majeure partie du temps de développement d’un projet. Elle doit être envisagé dès la conception :

- Code bien architecturé
- Code évolutif et robuste
- Utilisation de bonnes méthodologies (Agile, Cycle en V)
- Stratégie de test automatisé
- Utilisation de processus de Devops : CI/CD

Présentation de Java

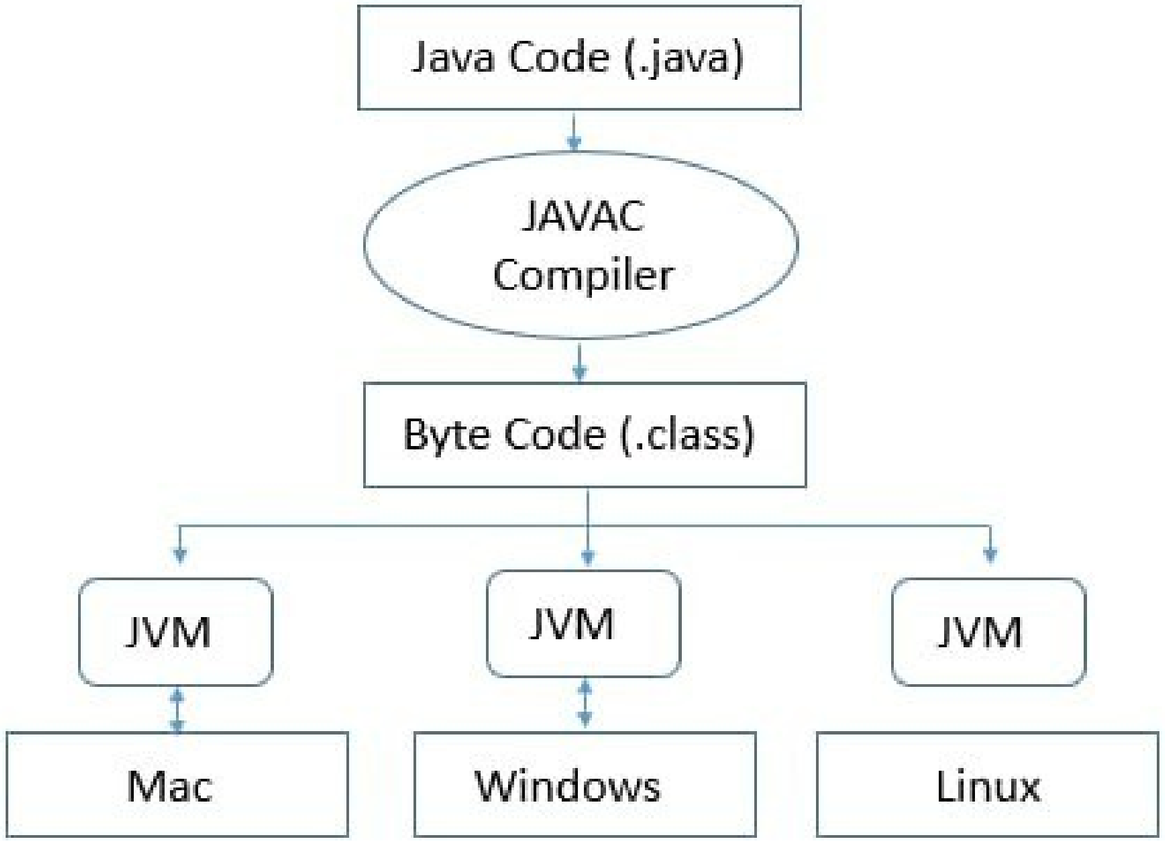
Dany Siriphol – CODA 1 ère année - 2025

Présentation de Java

Langage de programmation **haut niveau, orientée objet** programmation crée en 1995 par Sun Microsystems

- Portable : write once, run everywhere
- Langage compilé et exécuté dans une Machine Virtuel (JVM)
- Langage typé
- Gestion automatique de la mémoire (Garbage Collector)
- Multitâche
- Grand écosystème
- Version actuelle LTS : Java 21

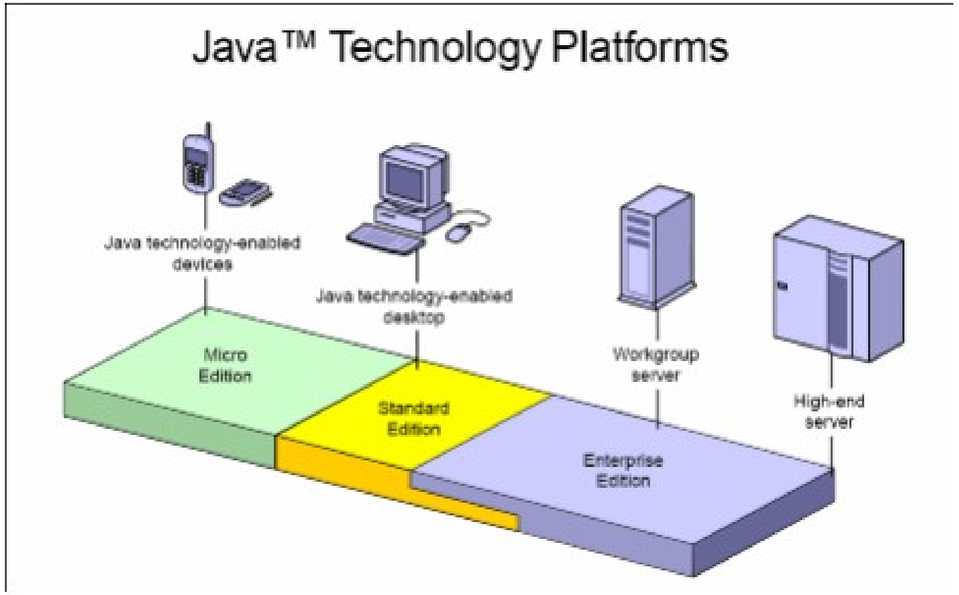
Java Virtual Machine (JVM)



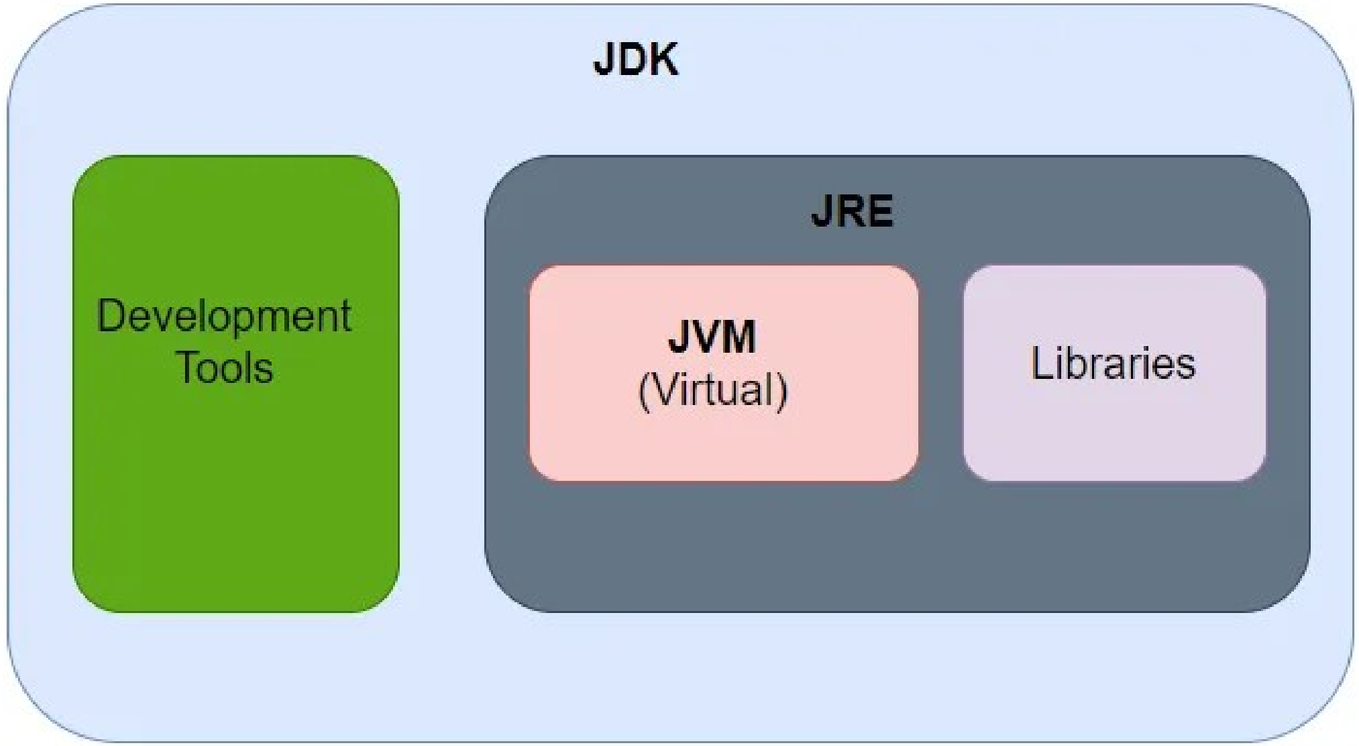
Plateforme Java

- J2SE (Standard Edition) – Pour les applications de bureau.
- Jakarta EE anciennement J2EE (Enterprise Edition) – Pour les applications web et d'entreprise.
- J2ME (Micro Edition) / Java SE Embedded – Pour les logiciels embarqués

- Android SDK – Développement Mobile



Plateforme Java



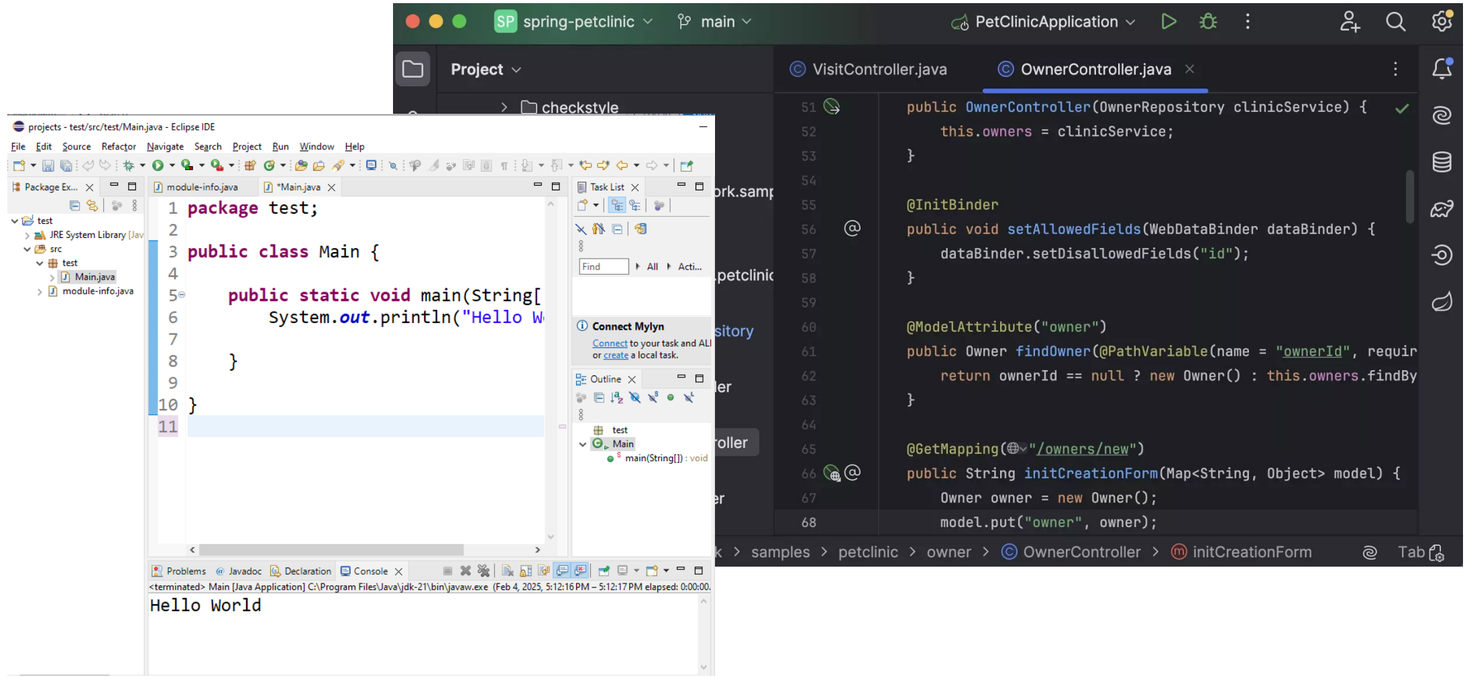
Outils nécessaires

Java JDK/SDK

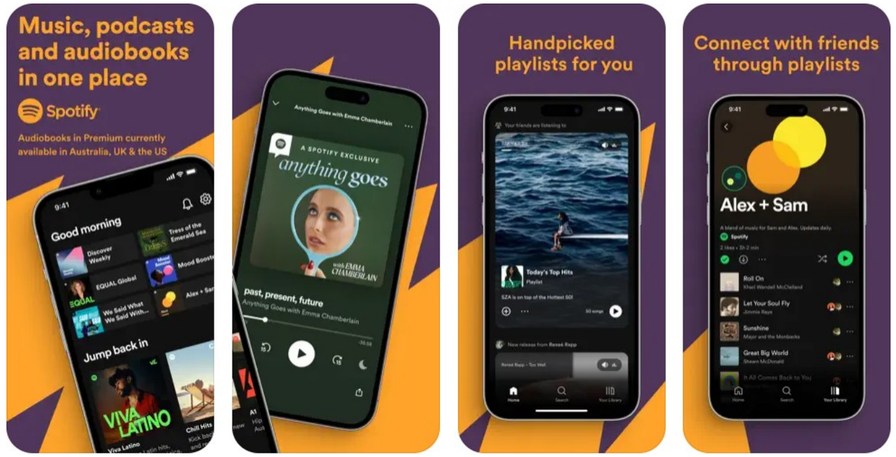
https://www.oracle.com/fr/java/technologies/downloads

IDE

IntelliJ IDEA, Eclipse



Utilisation de Java



Mobile Android

Web : J2EE, spring boot

Gaming

Programmation embarqué : Iot, Electronique, modem

PayPal

Application d'entreprise, systeme bancaire

Présentation du module

15 jours de cours

Apprendre un langage orienté objet : Java
Comprendre le fonctionnement de Java et d’une machine virtuelle
Comprendre la Programmation Orientée Objet et ses notions

Architecturer correctement son code

- Créer des applications modulaires et flexibles
- Bien utiliser les concepts en POO
- Savoir créer un code utilisable professionnellement

Journée type

Présentation des notions
Exercices
Correction
Note : **Ce support de cours est inutile**. Les connaissances se trouvent dans les exercices et votre savoir faire. Ne soyez pas passif pendant la journée

Contrôle des connaissances

Examen sur table: 30%

2h en fin de module

Projet à réaliser en binôme: 70%

À faire en 1 semaine

Lancer un programme Java

Dany Siriphol – CODA 1 ère année - 2026

Exemple de programme Java

```java
package test;

public class Main {

    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Executer un code Java

Compilation

```
javac ./test/Main.java
```

Execution

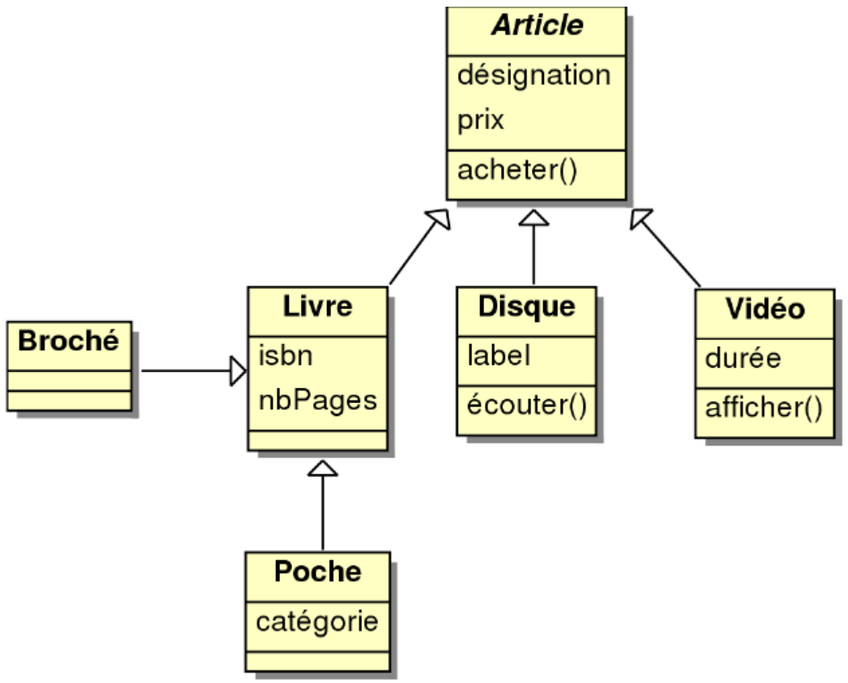
```
java test.Main
```

Présentation de la Programmation Orientée Objet (POO)

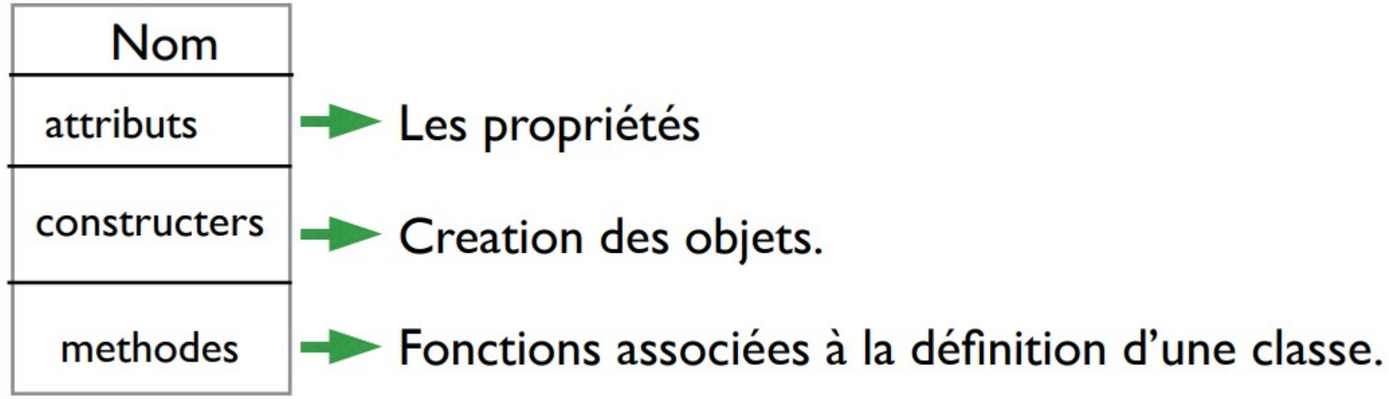
Dany Siriphol – CODA 1 ère année - 2026

Qu’est ce que la Programmation Orientée Objet ?

Paradigme de programmation qui consiste à structurer ses programmes la sous forme d'**objets** interagissant entre eux.



POO : Classe


Nom
attributs
Les propriétés
constructers
Creation des objets.
methodes
Fonctions associées à la définition d'une classe.

POO : Classe

Classe

Une classe est un type défini par le développeur pour représenter un concept.

Une classe est composé d'un ensemble de données (propriétés ou **attributs**) et de fonctions (**methodes**) qui agissent dessus

```
public class Person {
    ...
}
```

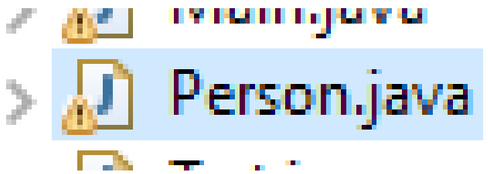
POO : Classe

Classe

En Java votre code doit obligatoirement se trouver dans une classe. Il n’est pas possible d’écrire ou d’exécuter du code se trouvant en dehors

Chaque classe doit se trouver dans un fichier portant le même nom

Un fichier ne peut contenir qu’une seule classe




```
public class Person {
    ...
}
```

POO : Attributs de classe

Attributs ou propriétés

Les classes sont composé d’attributs ou propriétés qui servent à représenter les caractéristiques d’une classe

En Java, les attributs doivent être défini au début de la classe

```
public class Person {

    private String firstname;
    private String lastname;
    private int age;
}
```

POO : Instances et constructeurs

Instances

Une fois la classe définie, il faut créer des objets de cette classe pour pouvoir les utiliser. On appelle ces objets des **instances** d'une classe

Il faut d'abord définir une fonction que l'on appelle le constructeur : C'est une fonction qui porte le mème nom que la classe et qui sert à définir les valeurs par défaut des propriétés

Ensuite on peut créer des objets de cette classe grâce au mot clé **new**. On appelle cela créer des instances ou instanciation.
L'instanciation d'une classe appelle la fonction **constructeur**.

```java
public class Person {

    private String firstname;
    private String lastname;
    private int age;

    public Person(String firstname, String lastname) {
        this.firstname = firstname;
        this.lastname = lastname;
    }

}
```

```java
Person p = new Person("John", "Doe");

System.out.println("Hello " + p.firstname);
```

POO : Methodes

Méthodes

Une classe peut contenir des fonctions appelées méthodes : Leur but est d'attacher des fonctionnalités aux classes.

Pour utiliser une méthode, **il faut les appeler depuis une instance de la classe**

```java
public class Person {

    public String firstname;
    public String lastname;
    public int age;

    ...

    public void getOlder() {
        this.age ++;
    }
}
```

```java
Person p = new Person("John", "Doe");
p.getOlder();

System.out.println("Vous avez " + p.age + " ans");
```

POO : This

this

Les instances d'une classe sont indépendantes les unes des autres.

Le mot clé **this** permet d'accéder aux éléments de l'instance courante

```
public String firstname;
public String lastname;
public int age;

public Person(String firstname, String lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
    this.age = 25;
}

public void getOlder() {
    this.age ++;
}
```

Java : Syntaxe

Dany Siriphol – CODA 1 ère année - 2026

Syntaxe de Java

Une instruction en C se termine toujours par un point virgule (;)

```
System.out.println("Hello World");
```

Votre code doit toujours se trouver une classe possédant une méthode public static void main (…)

```
public class Main {

    public static void main(String[] args) {
        System.out.println("Hello World");

    }

}
```

Variables

Syntaxe

Java est un langage typé et il faut indiquer le type des variables lors de la déclaration

```
int i = 5;
double number = 5.15;
float number2 = 20.012f;
String text = "Hello";
```

Structures de contrôle

Syntaxe

Java possède toutes les structures de contrôle classiques : conditions, boucles, switch/case

```
if ( condition ) {
    // ...
} else {
    // ...
}
```

```
while ( condition ) {
    // ...
}
```

```
for( int i = 0; i < 10 ; i++ ) {

}
```

Type primitifs

Data Type	Description
`byte`	Stores whole numbers from -128 to 127
`short`	Stores whole numbers from -32,768 to 32,767
`int`	Stores whole numbers from -2,147,483,648 to 2,147,483,647
`long`	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`float`	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
`double`	Stores fractional numbers. Sufficient for storing 15 to 16 decimal digits
`boolean`	Stores true or false values
`char`	Stores a single character/letter or ASCII values

String et concaténation

```
String greeting = "Hello";

String text = greeting + "World";

String text2 = "Hello " + 4;
```

Remarques

Écrire un code clair

Utiliser des noms parlant et clair pour tout votre code :

- Variable
- Nom de classe
- Nom de méthode

Votre code doit être écrit entièrement en anglais

Écrire un code clair

Nous utiliseront la convention **CamelCase** pour ce cours

```
public class Person {
```

```
public void getOlder()
```

Avertissement : ChatGPT ou autres IA génératives

L'objectif de ce cours est de vous faire assimiler les notions fondamentales de POO et de construire correctement vos programmes

Ce n’est pas le résultat qui compte mais la façon d’y arriver. Être programmeur c’est d’abord faire des choix. Ne laissez pas un outil les faire à votre place et surtout ne devenez pas dépendant d’eux.

Vous pouvez vous aider d’une IA générative pour mieux comprendre une notion ou comme aide occasionnelle mais elle ne doit <u>jamais</u> se substituer à votre réflexion

Exercices